In [1]:
```
import pandas as pd
data = pd.read_csv("Flipkart_data.csv")
data.head()
```

Out[1]:

	review	rating
0	It was nice produt. I like it's design a lot. ...	5
1	awesome sound....very pretty to see this nd th...	5
2	awesome sound quality. pros 7-8 hrs of battery...	4
3	I think it is such a good product not only as ...	5
4	awesome bass sound quality very good bettary l...	5

In [2]:
```
data.isnull().sum()          #check missing values
```

Out[2]:
```
review    0
rating    0
dtype: int64
```

In [3]:
```
data.shape                      #check dataset size
```

Out[3]:
```
(9976, 2)
```

In [4]:
```
data.columns                   # check column names
```

Out[4]:
```
Index(['review', 'rating'], dtype='str')
```

In [5]:
```
def sentiment(rating):            # Convert rating to sentiment
    if rating >=4:
       return "Positive"
    elif rating == 3:
       return "Neutral"
    else:
       return "Negative"
data["sentiment"] = data["rating"].apply(sentiment)
data.head()
```

Out[5]:

	review	rating	sentiment
0	It was nice produt. I like it's design a lot. ...	5	Positive
1	awesome sound....very pretty to see this nd th...	5	Positive
2	awesome sound quality. pros 7-8 hrs of battery...	4	Positive
3	I think it is such a good product not only as ...	5	Positive
4	awesome bass sound quality very good bettary l...	5	Positive

In [6]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
data = pd.read_csv("flipkart_data.csv")                  #load the dataset
data.head()
def sentiment(rating):                                   # Convert rating to sentim
    if rating >=4:
        return "Positive"
    elif rating == 3:
         return "Neutral"
    else:
         return "Negative"
data["sentiment"] = data["rating"].apply(sentiment)
X = data["review"]                                         # create Train-test spli
y = data["sentiment"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_stat
vectorizer = TfidfVectorizer()                              #  Create TF-IDF
X_train_vec = vectorizer. fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
model = MultinomialNB()                                     # Create Train model
model.fit(X_train_vec, y_train)
pred = model.predict(X_test_vec)                            # Create Accuracy
print("Accuracy:", accuracy_score(y_test,pred))
```

```
Accuracy: 0.8276553106212425
```

In [ ]:

```
Apply Naive Bayes  model
```

In [8]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
data = pd.read_csv("flipkart_data.csv")                  #load the dataset
data.head()
def sentiment(rating):                                   # Create sentiment column
    if rating >=4:
        return "Positive"
    elif rating == 3:
         return "Neutral"
    else:
         return "Negative"
data["sentiment"] = data["rating"].apply(sentiment)
X = data["review"]                                         # create Train-test spli
y = data["sentiment"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_stat
vectorizer = TfidfVectorizer()                              #  Create TF-IDF
X_train_vec = vectorizer. fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
model = MultinomialNB()                                     # Create Train model
model.fit(X_train_vec, y_train)
pred = model.predict(X_test_vec)
review = ["This product is very good and amazing"]           # Testing the model
review_vec = vectorizer.transform(review)
prediction = model.predict(review_vec)
print("Sentiment:", prediction[0])
```

```
Sentiment: Positive
```

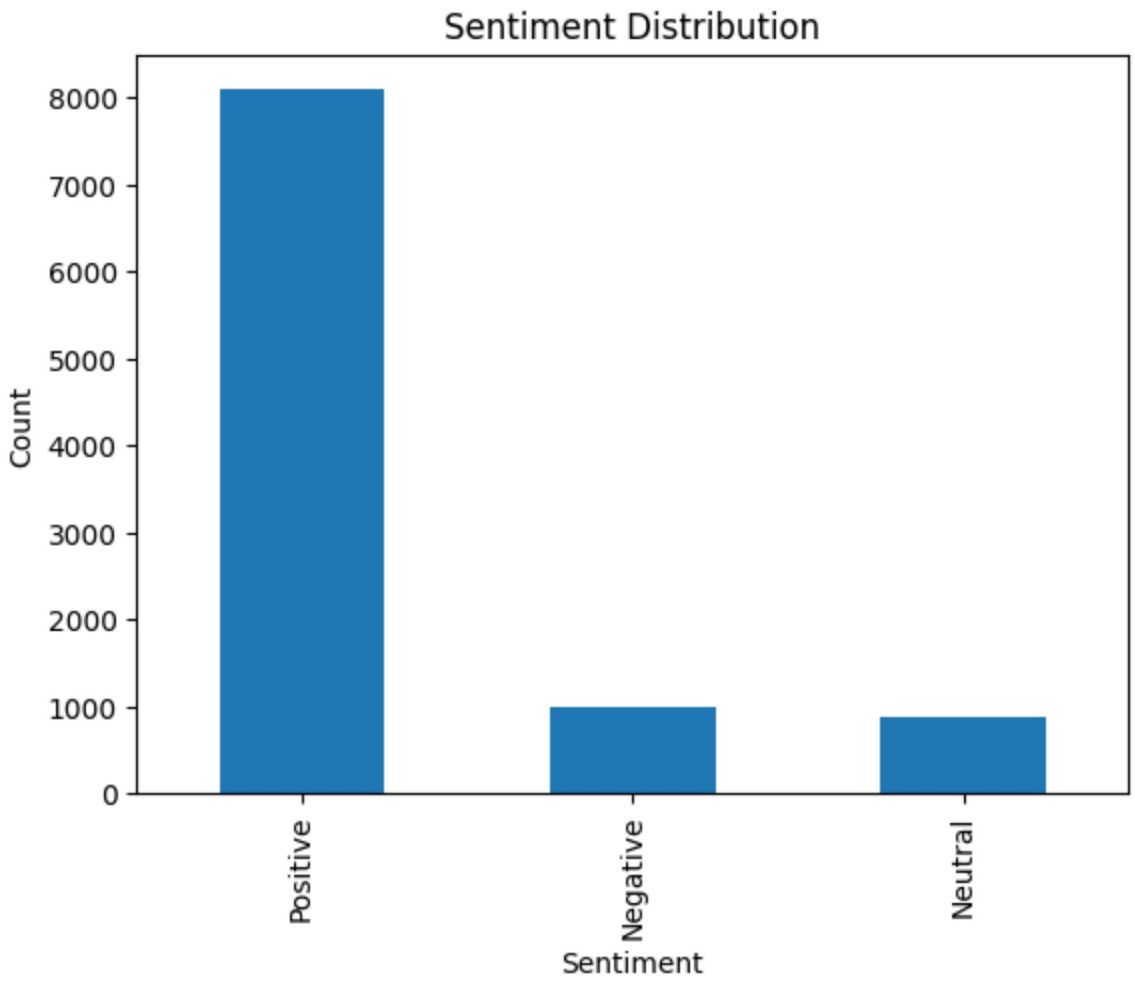
In [ ]:
```
Check Accuracy
```

In [14]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
data = pd.read_csv("flipkart_data.csv")                  #load the dataset
data.head()
def sentiment(rating):                                   # Create sentiment column
    if rating >=4:
        return "Positive"
    elif rating == 3:
         return "Neutral"
    else:
         return "Negative"
data["sentiment"] = data["rating"].apply(sentiment)
X = data["review"]                                         # create Train-test spli
y = data["sentiment"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_stat
vectorizer = TfidfVectorizer()                              #  Create TF-IDF
X_train_vec = vectorizer. fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
model = MultinomialNB()                                     # Create Train model
model.fit(X_train_vec, y_train)
pred = model.predict(X_test_vec)                            # Create Accuracy
print("Accuracy:", accuracy_score(y_test,pred))
print(confusion_matrix(y_test, pred))
```

```
Accuracy: 0.8276553106212425
[[  12    0  188]
 [   0    0  156]
 [   0    0 1640]]
```

In [ ]:
```
apply graph model or better understanding
```

In [15]:
```
import matplotlib.pyplot as plt
data["sentiment"].value_counts().plot(kind="bar")
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```


Sentiment Distribution
8000
7000
6000
5000
4000
3000
2000
1000
0
Count
Positive
Negative
Neutral
Sentiment

```
In [ ]:
```